

quizz-cours-IA — Rapport final

Module 11 — AI Agent Lab — Capstone Project

Auteur : Boukhatem

Date : Mai 2026

\newpage

1. Résumé exécutif

`quizz-cours-IA` est un **générateur de quiz pédagogiques multi-agents** conçu pour fonctionner **entièrement en local** via Ollama, sans aucune dépendance à un service cloud. Le projet répond aux exigences du Module 11 du AI Agent Lab : LLM local obligatoire, RAG local, dockerisation complète, sécurité documentée et démonstration reproductible.

Cas d'usage couvert : un enseignant ou un étudiant importe un support de cours (PDF, DOCX, TXT, MD) dans l'application. L'agent indexe le document, puis répond aux questions ou génère des quiz QCM à la demande, en citant systématiquement les sources (fichier + numéro de page).

Apports clés :

- Pipeline multi-agents LangGraph (router → RAG/tools → finalizer)
- LLM local Ollama avec sélecteur dynamique de modèle dans l'UI
- Ancrage strict des réponses dans le document (RAG) avec citations
- Validation pydantic des sorties LLM + récupération de JSON tronqué
- Frontend React/Tailwind + API FastAPI + Vector DB ChromaDB
- Gouvernance : prompt-injection guard testé, mémoire conversationnelle bornée

Démarrage en une commande : `docker compose up --build`.

\newpage

2. Contexte et problématique

2.1 Le problème métier

Préparer un quiz pédagogique de 10 questions à partir d'un support de cours prend en moyenne **30 à 45 minutes** à un enseignant : il faut lire, isoler les concepts clés, formuler des questions équilibrées, proposer des options plausibles et justifier la bonne réponse. Cette tâche est répétitive et peu valorisée alors qu'elle conditionne l'évaluation des étudiants.

Côté étudiant, l'**auto-évaluation** sur un support de cours est limitée par l'absence d'outils capables de générer des quiz à partir d'un document arbitraire — les annales se répètent, et les plateformes en ligne (Kahoot, Quizlet) demandent une saisie manuelle.

2.2 Pourquoi un agent IA local ?

Trois exigences techniques convergent vers une architecture multi-agents locale :

1. **Ancrage documentaire** (RAG) : les questions doivent porter sur le contenu réel du support, pas les connaissances générales du LLM.
2. **Décisions contextuelles** : selon la requête, le pipeline doit choisir entre RAG, outils externes ou les deux.
3. **Sortie structurée garantie** : le quiz est consommé par une UI typée — pas de génération texte libre.

L'exigence « local » découle de deux contraintes :

- **Souveraineté des données** : les supports de cours peuvent contenir des éléments confidentiels (sujets d'examen, polycopiés privés).
- **Conformité au Module 11** : « LLM local obligatoire — pas de dépendance à OpenAI ou API cloud ».

2.3 Objectifs du projet

Objectif	Mesure de réussite
Lancement reproductible	<code>docker compose up</code> fonctionne sur Linux + macOS/WSL
100 % local	Aucune sortie réseau vers domaine LLM cloud
Quiz ancré	Toutes les questions sourcées ont une citation <code>fichier:page</code> valide

Objectif	Mesure de réussite
Performance acceptable	Quiz 5 questions < 60s en CPU pur sur modèle 3B
Sécurité	Détection des prompt-injections testée (<code>pytest</code>)

\newpage

3. Analyse du besoin et cahier des charges

Le cahier des charges complet est présenté dans [docs/specifications.md](#). Synthèse des éléments-clés ci-dessous.

3.1 Public cible

Persona	Besoin principal
Enseignant	Générer rapidement des quiz d'évaluation depuis ses supports
Étudiant	S'auto-évaluer sur un cours par des quiz variés à la demande
Responsable formation	Industrialiser la production de QCM pour LMS

3.2 Fonctionnalités prioritaires (Must)

- F1 — Ingestion PDF/DOCX/TXT/MD via UI ou CLI
- F2 — Indexation vectorielle avec embeddings locaux
- F3 — Génération de quiz QCM et questions ouvertes (1 à 20 questions)
- F4 — Sélection dynamique du modèle LLM local
- F5 — Citations source `fichier:page` sur chaque question
- F7 — Reset complet du store et de la mémoire
- F8 — Réinitialisation automatique du store à chaque nouvel import
- F9 — Détection et neutralisation des prompt-injections
- F12 — API REST documentée (Swagger UI)

3.3 Contraintes imposées par le module

Contrainte	Implémentation
Docker obligatoire	<code>docker-compose.yml</code> orchestre tous les services
LLM local obligatoire	Service <code>ollama</code> + <code>llama3.2:3b</code> par défaut
RAG local	ChromaDB + <code>sentence-transformers/all-MiniLM-L6-v2</code>
Volumes persistants	<code>chroma_data</code> , <code>ollama_models</code> , <code>hf_cache</code>
Réplicabilité	Aucun chemin absolu, aucun OS-spécifique

3.4 Valeur ajoutée

Solution existante	Limitation	Apport quizz-cours-IA
Kahoot / Quizlet	Pas de génération depuis document	Ingestion PDF/DOCX native
ChatGPT « génère un quiz »	Pas d'ancrage, pas de source	RAG strict + citations
Plugins LMS (Moodle Quiz Gen)	Cloud-only, données envoyées dehors	100 % local
Pipelines artisanaux LangChain	Pas d'UI, pas de garanties de schéma	Frontend + pydantic

\newpage

4. Conception de l'architecture

L'architecture détaillée et les diagrammes (Mermaid) sont dans [docs/architecture.md](#). Vue synthétique ici.

4.1 Vue d'ensemble en couches

Client : React + Vite + Tailwind, servi par nginx
API : FastAPI + uvicorn, 7 endpoints REST
Orchestration : LangGraph (4 nœuds + état typé) router → {rag tools} → finalizer
Outils : quiz_generator, web_search (DuckDuckGo)
Données : ChromaDB + sentence-transformers (local)
LLM : Ollama (llama3.2:3b par défaut)
Gouvernance : prompt_guard + mémoire bornée + logs INFO

4.2 Diagrammes produits

Tous les diagrammes sont rendus en Mermaid dans [docs/architecture.md](#) :

1. **Cas d'utilisation** — acteurs (Enseignant, Étudiant) × 5 cas d'usage
2. **Composants** — 7 couches logiques avec leurs interactions
3. **Séquence** — génération d'un quiz pas-à-pas (UI → API → LangGraph → Ollama)
4. **Flux RAG** — ingestion (loader → chunker → embedder → upsert) puis recherche (query → embed → top-k → contexte → LLM)
5. **Déploiement Docker** — 5 conteneurs + 3 volumes + réseau Docker default

4.3 Décisions d'architecture clés

Décision	Justification
LangGraph plutôt que <code>SequentialChain</code>	Routage conditionnel natif, état TypedDict, traçabilité par nœud

Décision	Justification
Ollama plutôt que llama-cpp-python embarqué	Service séparé → couplage faible, mutualisation des modèles
ChromaDB plutôt que Qdrant/Weaviate	API simple, fallback PersistentClient sans serveur
Embeddings locaux plutôt qu'API cloud	Zéro dépendance externe, conformité au lab
pydantic v2 pour les sorties LLM	Garanties de schéma, retry sur erreur de validation
FastAPI plutôt que Flask	Validation auto, OpenAPI gratuit, async-ready

\newpage

5. Implémentation technique

5.1 Stack complète

Couche	Technologie	Version
Frontend	React + Vite + TypeScript + Tailwind	React 18, Vite 5
Serveur web	nginx Alpine	latest
Backend	FastAPI + uvicorn	0.115+
Orchestration	LangGraph	0.2+
Validation	pydantic v2	2.x
LLM	Ollama	latest
Modèle par défaut	llama3.2:3b	~2 Go
Modèles alternatifs	gemma2:2b, qwen2.5:7b	allowlist
Embeddings	sentence-transformers/all-MiniLM-L6-v2	384 dims
Vector DB	ChromaDB	0.5.20
Recherche web	duckduckgo-search	latest
HTTP client	httpx	sync, timeout 300s
Tests	pytest	8.x

5.2 Structure du code

```
app/
├─ api.py           # Endpoints FastAPI + lifecycle
├─ main.py          # CLI Rich (REPL)
├─ cli.py           # Boucle REPL et commandes
├─ config.py        # Settings Pydantic + .env
├─ llm.py           # OllamaClient (chat, set_model, JSON mode)
├─ graph.py         # Construction du LangGraph
├─ router.py        # Nœud router (heuristique + LLM tie-breaker)
├─ state.py         # GraphState (TypedDict)
├─ memory.py        # ConversationMemory bornée
├─ rag/
│   └─ loader.py    # Loaders PDF/DOCX/TXT/MD
│   └─ chunker.py   # Découpage en chunks
│   └─ embeddings.py # sentence-transformers
│   └─ vectorstore.py # ChromaDB + fallback persistant
│   └─ agent.py     # rag_node
└─ tools/
```

```

├── web_search.py      # DuckDuckGo
├── quiz_generator.py  # Génération quiz + salvage JSON
├── agent.py           # tools_node
└── security/
    └── prompt_guard.py # Détection injections

web/
├── src/
│   ├── App.tsx        # Composant racine
│   ├── api.ts         # Client REST
│   ├── types.ts       # Types partagés
│   └── components/
│       ├── TopBar.tsx
│       ├── MessageBubble.tsx
│       ├── ThoughtList.tsx
│       └── QuizCard.tsx
└── nginx.conf         # Proxy /api → backend

tests/
├── test_rag.py
├── test_security.py
└── test_tools.py

```

5.3 Docker compose

5 services orchestrés :

Service	Image	Rôle
chromadb	chromadb/chroma: 0.5.20	Vector DB avec healthcheck
ollama	ollama/ollama:latest	Serveur LLM local
ollama-init	ollama/ollama:latest	Pull du modèle par défaut (llama3.2:3b) puis exit
api	build local	FastAPI + uvicorn
web	build local	nginx + bundle React
cli	reuse image api	Profile optionnel, REPL Rich

Dépendances : api attend que chromadb et ollama soient healthy ; ollama-init attend que ollama soit healthy puis pull le modèle.

5.4 API REST

Méthode	Endpoint	Description
GET	/api/health	Liveness check

Méthode	Endpoint	Description
GET	/api/status	Chunks, mémoire, modèle actif, modèles disponibles
GET	/api/models	Liste des modèles autorisés
POST	/api/models	Change le modèle actif
POST	/api/query	{query} → {thoughts, final_answer, quiz?}
POST	/api/ingest	multipart file → {filename, chunks}
POST	/api/reset	Vide le store et la mémoire

Documentation interactive : <http://localhost:8000/docs> (Swagger UI auto).

\newpage

6. Intégration RAG + Outils + Planification + Gouvernance

6.1 RAG

Ingestion : `app/rag/loader.py` détecte l'extension (PDF/DOCX/TXT/MD) et charge le document en pages. `chunker.py` découpe en chunks d'environ 800 caractères avec overlap pour préserver le contexte. Les métadonnées préservées sont `source` (nom de fichier) et `page` (numéro).

Indexation : `embeddings.py` calcule les vecteurs via `sentence-transformers/all-MiniLM-L6-v2` (384 dimensions). `vectorstore.py` fait un `upsert` dans la collection ChromaDB (idempotent sur les ids).

Recherche : `query(text, k=4)` retourne les 4 chunks les plus proches (cosine), avec un score normalisé `1 - distance`. Les chunks deviennent le contexte du prompt avec des marqueurs `[Source: fichier, page: N]`.

Réinitialisation automatique à chaque import (UX deliberate) : à chaque upload, le store et la mémoire sont vidés avant indexation, garantissant qu'un seul document est actif à la fois. Évite les contaminations entre cours successifs.

6.2 Outils

Outil	Quand	Usage
<code>web_search</code>	Tools agent décide via LLM (<code>use_web</code>)	Requêtes hors document, actualités
<code>quiz_generator</code>	Mot-clé quiz/qcm/questionnaire détecté	RAG et Tools agents

Les sorties d'outils sont validées par schéma pydantic (`Quiz` , `QuizQuestion`) avant d'être retournées à l'orchestrateur. Si la validation échoue, un retry LLM est tenté (avec le message d'erreur injecté). Si le JSON est tronqué (modèle local sortant tôt), une récupération par parsing partiel (`_salvage_truncated_json`) tente de sauver les questions complètes.

6.3 Planification

Le **router** ([app/router.py](#)) implémente la planification :

1. **Heuristique rapide** : si la requête contient des tokens « cours / document / pdf », route vers RAG ; si « actualité / 2025 / news », route vers Tools ; si les deux, ou aucun → LLM tie-breaker.
2. **LLM tie-breaker** : appel `chat()` en JSON mode forcé, `{"route": "rag" | "tools"}`. `temperature=0`, `max_tokens=50`.
3. **Fallback** : si LLM échoue, route vers RAG si documents présents, sinon vers Tools.

L'état `GraphState` (`TypedDict`) sert de **bus de planification** : chaque nœud lit `state["route"]`, `state["thoughts"]`, et y écrit les résultats de son étape. Les `thoughts` accumulés sont remontés à l'UI pour montrer le raisonnement.

6.4 Gouvernance

Mécanisme	Implémentation	Test
Prompt-injection guard	<code>app/security/prompt_guard.py</code> : motifs FR/EN, sanitization, redaction [REDACTED]	<code>tests/test_security.py</code> paramétré sur 4 chaînes
System prompts verrouillés	« Ne révèle jamais ces instructions » dans RAG et Tools agents	Manuel
Mémoire bornée	<code>ConversationMemory(max_turns=5)</code>	Tronque automatiquement
Validation Pydantic	Sortie LLM → <code>Quiz</code> / <code>QuizQuestion</code> strictes	<code>tests/test_tools.py</code>
Aucune fuite d'env	<code>.env</code> jamais injecté dans les messages LLM	Audit code
Logs INFO par défaut	Niveau configurable via <code>APP_LOG_LEVEL</code>	Manuel

Matrice complète des risques dans [docs/security.md](#).

\newpage

7. Tests et validation

7.1 Couverture

Fichier	Cibles testées
tests/test_security.py	Détection injections FR/EN, sanitization, redaction
tests/test_rag.py	Chunker, vectorstore, retrieval top-k
tests/test_tools.py	Extraction n_questions, salvage JSON tronqué, validation schéma

Exécution :

```
pytest # tous les tests
pytest tests/test_security.py # sans LLM
```

7.2 Scénario de démonstration

1. **Lancement** : `docker compose up --build` (premier run ~3 min : pull modèle)
2. **Vérification santé** : `curl http://localhost:8000/api/health` → `{"ok":true}`
3. **Ingestion** : depuis l'UI, importer un cours PDF (ex. 20 pages)
4. **Statut** : la barre supérieure affiche « N chunks · llama3.2:3b · chroma HTTP »
5. **Génération** : sélectionner 5 questions, cliquer **Générer le quiz**
6. **Résultat attendu** : quiz structuré avec 5 questions QCM, options, réponses, explications et citations [Source: cours.pdf, page: X]
7. **Changement de modèle** : sélecteur en haut → `gemma2:2b` (après `docker compose exec ollama ollama pull gemma2:2b`)
8. **Reset** : bouton **Réinitialiser** → store et mémoire vidés

\newpage

8. Résultats

8.1 Conformité aux exigences techniques

Exigence Module 11	Statut
Docker obligatoire	✓ docker-compose.yml complet
LLM local obligatoire (Ollama)	✓ Service ollama + modèle local
RAG local	✓ ChromaDB + sentence-transformers local
Volume persistant	✓ 3 volumes nommés
Répliquabilité	✓ Aucun chemin absolu / OS-spécifique
Frontend UI locale	✓ React + Tailwind
Multi-agents	✓ LangGraph (router, rag, tools, finalizer)
Documentation Docker + .env	✓ README + .env.example

8.2 Performances mesurées

Scénario	Temps observé
Cold start (avec pull modèle)	2 à 5 min
Démarrage à chaud	< 10 s
Ingestion PDF 20 pages	3 à 5 s
Génération quiz 5 questions (CPU, 3B)	25 à 55 s
Génération quiz 10 questions (CPU, 3B)	50 à 90 s
Recherche RAG top-4	< 200 ms

8.3 Limites identifiées

- **Latence LLM en CPU pur** : pour un modèle 3B, ~10 tokens/s. Acceptable en démo, perfectible avec GPU ou modèle quantisé Q4.
- **Mono-corpus** : un seul document actif à la fois (décision UX).
- **Pas de streaming** des réponses (latence perçue plus haute).

\newpage

9. Innovation et esprit critique

9.1 Apports originaux

1. **Salvage de JSON tronqué** ([quiz_generator.py:80-126](#))
un parser tolérant qui récupère les questions complètes même si le modèle local s'arrête en plein milieu d'un tableau. Critique sur petits modèles où la fenêtre de génération peut être courte.
2. **Réinitialisation à l'import** : au lieu d'accumuler les corpus (comportement par défaut RAG), chaque nouvel upload remplace le précédent. Réflexion UX : pour un usage « un cours = un quiz », l'accumulation est une source de bruit, pas une feature.
3. **Allowlist de modèles côté config + UI** : pas de saisie libre → l'utilisateur ne peut sélectionner que des modèles pré-validés. Sécurité et UX renforcées.
4. **Migration Gemini → Ollama transparente** : grâce à l'interface `OllamaClient.chat()` identique à l'ancien `GeminiClient`, le code des 4 nœuds (router, rag, tools, quiz_generator) n'a **pas été modifié** lors de la bascule. Architecture découplée du fournisseur LLM.

9.2 Pistes d'amélioration

- **Mode streaming** SSE vers le frontend (latence perçue)
- **Mode GPU** Ollama via `deploy.resources.devices` du compose
- **Multi-corpus** avec sélecteur de document actif
- **Export PDF / Markdown** du quiz généré
- **Cache LRU** des couples (query, top-k chunks) pour démos répétées
- **Observabilité** : logs JSON structurés + endpoint `/api/audit`

\newpage

10. Reproductibilité et démonstration

10.1 Pré-requis

- Docker + Docker Compose v2
- 8 Go de RAM disponibles minimum
- 5 Go d'espace disque (modèle + image)
- Connexion internet uniquement pour le premier `pull`

10.2 Procédure

```
git clone <repo-url>
cd quizz-cours-IA
docker compose up --build
```

Premier démarrage : ~3 min pour le pull `llama3.2:3b` . Démarrages suivants : < 10 s grâce au volume `ollama_models` .

Accès :

- UI : <http://localhost:5173>
- API + Swagger : <http://localhost:8000/docs>
- Ollama : <http://localhost:11434>

10.3 Tests sur deux environnements

OS	Statut
Ubuntu 24.04 (cible primaire)	✓ Testé
Windows 11 + WSL2 Ubuntu	À tester
macOS 14 (Apple Silicon)	À tester

\newpage

11. Conclusion

`quizz-cours-IA` répond aux **8 critères de la grille officielle Module 11** en combinant :

- un **cas d'usage métier réel et utile** (éducation, génération de quiz),
- une **architecture multi-agents** documentée par 5 diagrammes Mermaid,
- une **conformité totale aux obligations techniques** (Docker, LLM local Ollama, RAG local, volumes persistants),
- une **gouvernance testée** (prompt-injection guard avec tests pytest),
- une **expérience utilisateur soignée** (frontend React + Tailwind, sélecteur de modèle, indicateurs de raisonnement),
- une **innovation technique** (salvage JSON tronqué, migration LLM transparente, allowlist).

Le projet est **reproductible en une commande** et démontrable de bout en bout, du `docker compose up` jusqu'au quiz généré avec sources.

Livrables fournis :

- Code source : dépôt Git
- Cahier des charges : `docs/specifications.md`
- Architecture + diagrammes : `docs/architecture.md`
- Sécurité : `docs/security.md`
- Rapport final : `docs/report.pdf` (ce document)
- README + `.env.example` : à la racine
- Tests : `tests/` (pytest)